Getting Started with LangChain

LangChain Framework Overview

LangChain is a framework that connects large language models with external tools, memory, and data sources. It enables developers to build applications that perform multi-step reasoning, use stored context, and interact with structured data or APIs efficiently. This connection allows language models to move beyond single prompts and create dynamic, workflow-driven experiences.

```
from langchain import LangChain

# Initialize LangChain with a large language model
langchain = LangChain(model_name="gpt-3.5-turbo")

# Connect external tools and data sources
langchain.connect(tool="text-summarizer")
langchain.connect(data_source="remote-database")

# Create a multi-step workflow
def process_input(input_data):
    # Step 1: Summarize the input
    summary = langchain.call_tool("text-summarizer", input_data)
    # Step 2: Store summary in the database
    langchain.store_data("remote-database", summary)

# Run workflow with user input
process_input("Lorem ipsum dolor sit amet...")
```

LangChain `PromptTemplate`

A `PromptTemplate` in LangChain allows you to create dynamic prompts by using placeholders for various inputs. This enables the creation of reusable templates that can adapt based on the context or topic provided. It's a convenient way to manage different prompt formats consistently across your application.

```
from langchain.prompts import
PromptTemplate

# Define a reusable prompt template
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain the importance of
{topic} in simple terms."
)

# Fill in the placeholder
prompt = template.format(topic="machine
learning")

print(prompt)
# Output: "Explain the importance of
machine learning in simple terms."
```

LangChain Wrappers

LLM wrappers are classes in LangChain that provide a common interface for interacting with different language model providers. They make it easier to switch between models, such as OpenAI, Anthropic, or Hugging Face, without changing the core application code.

```
from langchain import LLMWrapper

# Example of initializing a language
model
llm = LLMWrapper(model_name="openai-
gpt3")

# Interacting with the initialized model
response = llm.prompt("Translate 'hello'
to French")

print(response)  # Expected output:
'bonjour'
```

LangChain `OutputParser`

In LangChain, an `OutputParser` is a component that formats the model's raw text into structured output. It helps create clean, reliable results that can be used in the application or passed to the next step in a chain.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import
PromptTemplate
from langchain_core.output_parsers import
StrOutputParser

prompt = PromptTemplate(
    input_variables=["question"],
    template="Answer this question:
{question}"
)

model = ChatOpenAI(model="gpt-4")
parser = StrOutputParser()

chain = prompt | model | parser

response = chain.invoke({"question":
"What's the weather like today?"})
print(response)
```

LangChain `SequentialChain`

A `SequentialChain` in LangChain allows you to connect multiple chains, where each output serves as the next input, facilitating multi-step workflows seamlessly.

```
from langchain import SequentialChain

# Define individual chain functions

def first_chain(input_data):
    return input_data + " processed by first chain."


def second_chain(input_data):
    return input_data.upper()

# Set up the sequential chain
sequential_chain = SequentialChain([
    first_chain,
    second_chain
])

# Run the sequential chain
initial_input = "This is some input"
result = sequential_chain.run(initial_input)
print(result)
# Output: THIS IS SOME INPUT PROCESSED BY FIRST CHAIN.
```

LangChain Workflows

Chains in LangChain enable predefined workflows with a fixed sequence of steps, making them ideal for consistent and repeatable tasks. They execute logic that follows a reliable order of operations, ensuring predictability and efficiency in complex processes.

```
from langchain import LLMChain, OpenAI
from langchain.prompts import
PromptTemplate

# Define a simple prompt and model
prompt = PromptTemplate(
    input_variables=["task"],
    template="Explain the importance of
{task} in one sentence."
)
llm = OpenAI(model="gpt-4")

# Create and run a chain
chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run(task="data security")
print(result)
```

Stateless LLMs

Large language models operate without memory, analyzing each message in isolation. This means they don't remember previous prompts unless explicitly told. Their stateless nature can be advantageous for tasks that require unbiased and independent analysis of text.

```
from langchain.llms import OpenAI

# Initialize a language model
llm = OpenAI(model="gpt-4")

# Each prompt is processed independently
response_1 = llm("My name is Alex.")
response_2 = llm("What is my name?")

print(response_1)
print(response_2)
# The model won't remember "Alex" since
it has no memory by default.
```

LangChain `ConversationBufferMemory`

`ConversationBufferMemory` is a memory type in LangChain that stores the full conversation history exactly as written. It allows models to access and reference every previous message during an interaction.

```
from langchain.memory import
ConversationBufferMemory

# Initialize the memory buffer
dialogue_memory =
ConversationBufferMemory()

# Add conversation entries
dialogue_memory.add_memory("User: What is
LangChain?")
dialogue_memory.add_memory("AI: LangChain
is a framework designed to ...")
dialogue_memory.add_memory("User: How
does it handle memory?")

# Retrieve full conversation
dialogue_history =
dialogue_memory.get_memory()
print(dialogue_history)
# Output shows the complete dialogue
stored in memory.
```

LangChain Memory

`ConversationBufferWindowMemory` is a memory type in LangChain that retains only the most recent N messages from a conversation. It maintains a fixed-size sliding window of context, helping models stay focused on recent exchanges without storing the entire history.

```
from langchain.memory import
ConversationBufferWindowMemory

# Initialize memory with a window size of
5
memory =
ConversationBufferWindowMemory(window_siz
e=5)

# Simulated conversation
memory.add_message("User: How's the
weather?")
memory.add_message("Bot: It's sunny!")
memory.add_message("User: Great! Any
plans for today?")
memory.add_message("Bot: A walk in the
park sounds nice.")
memory.add_message("User: Enjoy the
sunshine!")
memory.add_message("Bot: Thank you!")

# Retrieve the current context
current_context = memory.get_context()
print(current_context)  # Outputs the
last 5 messages
```

LangChain Summary Memory

`ConversationSummaryMemory` is a memory type in LangChain that uses a language model to create and maintain a running summary of a conversation. It reduces token usage by condensing past exchanges while preserving essential context and key details.

```
from langchain.memory import
ConversationSummaryMemory
from langchain.llms import OpenAI

# Initialize memory with an LLM
summarizer
llm = OpenAI(model="gpt-4")
summary_memory =
ConversationSummaryMemory(llm=llm)

# Add conversation entries
summary_memory.add_memory("User: Can you
explain LangChain?")
summary_memory.add_memory("AI: LangChain
is a framework for building LLM-based
applications.")
summary_memory.add_memory("User:
Summarize what we discussed.")

# Retrieve the summarized conversation
conversation_summary =
summary_memory.get_memory()
print(conversation_summary)
# Output shows a concise summary of key
information.
```

LangChain Entity Tracking

`ConversationEntityMemory` is a memory type in LangChain that extracts and tracks specific entities such as names, places, and concepts mentioned throughout a conversation. It helps language models remember important details and maintain consistency across interactions.

```
from langchain.memory import
ConversationEntityMemory
from langchain.llms import OpenAI

# Initialize entity memory with an LLM
llm = OpenAI(model="gpt-4")
entity_memory =
ConversationEntityMemory(llm=llm)

# Add conversation entries
entity_memory.add_memory("User: My name
is Alex and I live in Paris.")
entity_memory.add_memory("AI: Nice to
meet you, Alex. How's Paris today?")

# Retrieve stored entities
entities = entity_memory.get_entities()
print(entities)
# Output includes tracked entities like
"Alex" and "Paris".
```

Print Share